

Background: Console and Hive

- **KubeStellar Console** — an AI-powered, multi-cluster Kubernetes dashboard whose value lives almost entirely in its **user interface**.
- **Hive** — an AI-agent orchestration system that dispatches GitHub issues/PRs to AI agents (Claude, Copilot, Gemini, Goose).

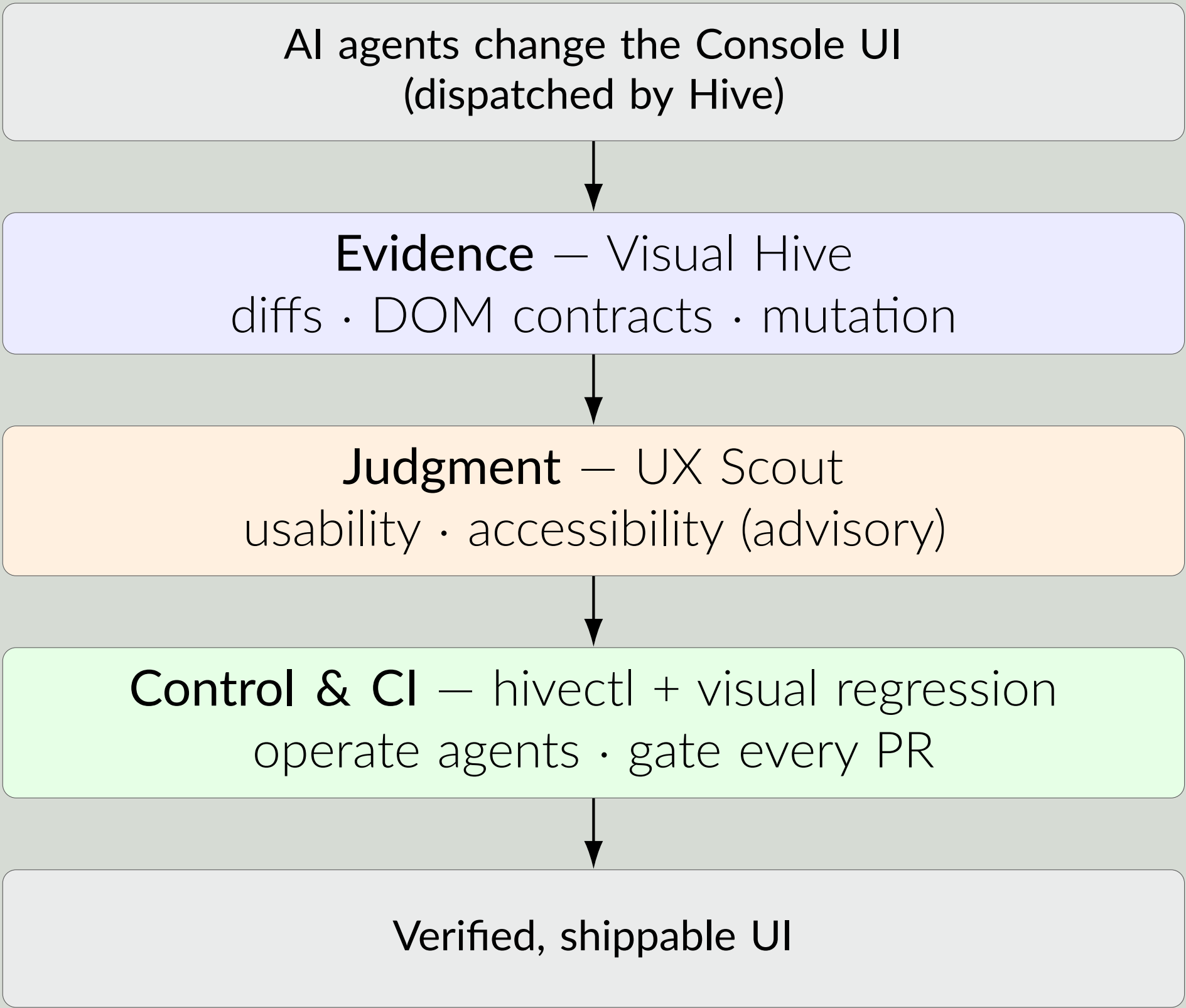
Hive lets AI agents *change* the Console faster than humans can review — so the Console needs a way to guarantee those changes don’t silently break the interface.

The UI/UX Challenge

As AI agents accelerate development, a quality gap opens that ordinary tests miss:

- **Regressions hide in pixels** — code compiles and tests pass, yet a card shifts, overflows, or renders the wrong state.
- **“Passing” is not “good”** — a screen can pass every pixel diff and still be confusing, inaccessible, or slow.
- **Failures are hard to reproduce** — a red CI run rarely says *which* route, component, or flow actually broke.
- **No safe control surface** — iterating on agents and quality policy from a browser is slow and error-prone.

Our Response: A Layered Quality Loop



Center column: **Visual Hive**. Right column: my work on the **judgment**, **control**, and **CI** layers.

References

Black, D. (1948). On the rationale of group decision-making. *Journal of Political Economy*, 56(1):23–34.

Sherif, M. and Hovland, C. I. (1961). *Social judgment: Assimilation and contrast effects in communication and attitude change*. Yale University Press.

My Project: Visual Hive

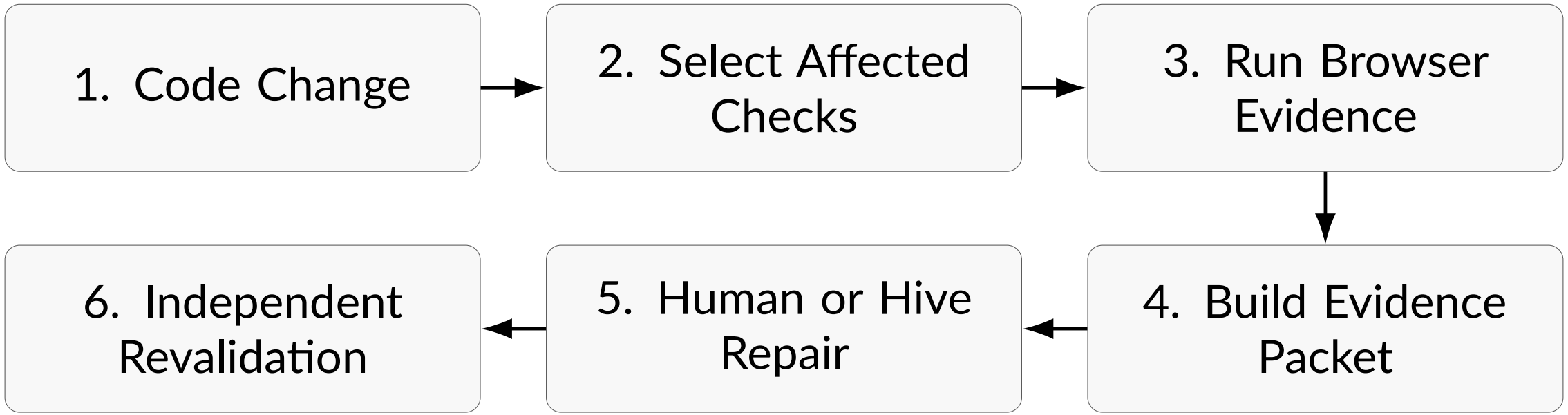
During my VSIP OSPO internship, I developed **Visual Hive**, a visual quality system designed around KubeStellar Console.

The goal was to make user-interface failures easier to detect, reproduce, and verify. Rather than relying on a screenshot difference alone, Visual Hive combines browser checks, visual comparison, interface-level assertions, mutation testing, and repository context to explain what failed and how the result should be validated.

What I Built

- **Change-aware test planning.** Maps modified files to affected components, routes, user flows, screenshots, and visual checks so that the most relevant tests run first.
- **Deterministic browser evidence.** Uses Playwright to check page structure, visible text, user interactions, screenshots, console errors, page errors, and network failures.
- **Live KubeStellar canaries.** Tests an authenticated KubeStellar Console instance connected to an Oracle Cloud Kubernetes cluster and compares the visible interface with application API responses and Kubernetes data.
- **Mutation-based test evaluation.** Introduces controlled interface faults to determine whether the existing tests can detect meaningful regressions.
- **Evidence and issue handoff.** Packages each finding with its affected route or component, screenshots, error evidence, reproduction command, and validation command for maintainers, GitHub, Hive, or another repair agent.
- **Safe CI automation.** Keeps pull-request checks read-only and limits credentials, issue publishing, branch creation, and repair actions to trusted workflows.

From Code Change to Verified Repair



Visual Hive links a failure to the affected interface, provides the evidence needed for repair, and reruns the relevant checks before the issue is considered resolved.

Impact

My work moved KubeStellar Console testing beyond isolated screenshot comparisons toward a repeatable visual feedback loop.

- Helped expose **interface and data inconsistencies** by comparing dashboard values and visible application state with API responses and Kubernetes ground truth.
- Distinguished **product regressions** from target-startup, environment, authentication, and test-configuration failures.
- Turned ambiguous CI failures into **structured, deduplicated issue candidates** with evidence and exact validation steps.
- Demonstrated that the workflow could operate from an **external consumer repository**, rather than only inside the Visual Hive codebase.
- Established a clear responsibility boundary: **Visual Hive evaluates and validates; Hive, maintainers, or other repair agents make code changes under project governance.**

UX Scout: A UX Judgment Agent

A portable **Hive agent** that reviews *user experience* where Visual Hive stops at “*did the UI change?*”

- **Advisory by design** — scores issues/PRs on Nielsen + WCAG 2.2 AA rubrics; emits JSON findings, never merges.
- **Honest evidence tiers** — asserts only what code proves; anything needing rendering becomes a *request*.
- **Low-noise & portable** — dedupes findings, silent on non-UI work, imports into any Hive.

hivectl: A Control Surface for Hive

A non-interactive CLI for a running Hive — operate the quality system from the terminal instead of a browser.

- **Operate agents** — import, kick, pause/resume, set prompt/model/backend (UX Scout is tuned entirely via **hivectl**).
- **Inspect & govern** — status, beads, knowledge, scheduler; stream live events as JSONL.
- **Safe by construction** — env-var tokens, **--yes** for destructive ops, typed exit codes for scripting.
- **Real auth** — browser-based GitHub device-flow login for self-hosted hives.

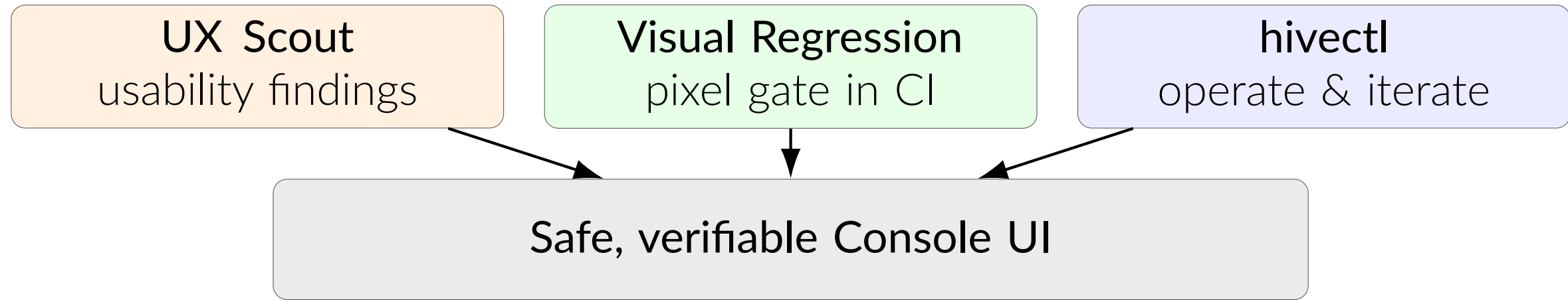
```
hivectl agent import --file ux-scout.yaml
hivectl agent kick quality --prompt "Review the work list"
```

Visual Regression for the Console UI

A Playwright suite that guards the Console UI on every UI-touching PR.

- **Committed baselines** — rendered on Linux to match CI; CI fails if a baseline is missing.
- **Storybook screenshot diff** — 18 components across Dark/Classic/Light themes, 1% pixel tolerance.
- **Layered gate** — runs beside build, lint, coverage, and AI-antipattern checks before merge.

How My Work Fits Together



Impact

Together these layers make a fast, AI-driven Console **verifiable**:

- **From “changed” to “usable”** — UX Scout adds a usability/accessibility judgment layer diffs cannot capture.
- **Regressions caught pre-merge** — committed visual baselines fail the PR on any unintended pixel change.
- **An operable quality system** — **hivectl** makes agents, prompts, and policy scriptable from CI.
- **Honest, low-noise findings** — advisory-only, evidence-tiered, and deduplicated, so it is safe across many repos.